

ΕΡΓΑΣΙΑ

Εργασία ανάπτυξης κώδικα

Τίμια Ψηφιακά Ζάρια.

Δύο διαδικτυακοί παίκτες (πχ ένας παίκτης και ο server) παίζουν ζάρια (από ένα ο καθένας). Κατά την εκτέλεση του παιχνιδιού αποφασίζεται το νικητήριο ενδεχόμενο (πχ κερδίζει το 3, κερδίζει όποιος φέρει πάνω από 4 κοκ)

[40%] Σχεδιάστε και υλοποιείτε ένα κρυπτογραφικό πρωτόκολλο που θα αποτρέπει ανέντιμη συμπεριφορά μεταξύ των παικτών και θα εξασφαλίζει δίκαια τον νικητή. Στην υλοποίηση: παράγεται από κάθε μέρος και πρώτα εμφανίζεται τοπικά στη διεπαφή χρήστη και εξυπηρέτη το αποτέλεσμα της ρίψης. Κατόπιν εκτελείται το κρυπτογραφικό πρωτόκολλο. Στην εκτέλεση αποφασίζεται το νικητήριο ενδεχόμενο. Στο τέλος, κάθε μέρος με βάση το πρωτόκολλο εμφανίζει στην διεπαφή κάθε μέρους αν είναι νικητής ή όχι.

[30%] Προσθέστε αυθεντικοποίηση server και από άκρο σε άκρο κρυπτογράφηση χρησιμοποιώντας το OpenSS. Δημιουργήστε Certificate Authority (CA) και ένα SSL certificate.

[30%] Επεκτείνετε την αυθεντικοποίηση και εισάγετε φόρμα εγγραφής χρήστη (όνομα, επίθετο, username και password) και φόρμα login με username και password. Δημιουργήστε σχεσιακή βάση δεδομένων (με όνομα GDPR) σε σύστημα ΒΔ δεδομένων και πίνακα users στη βάση που αποθηκεύει τα απαραίτητα δεδομένα για τη βασική λειτουργία Μηχανισμού Login με πεδία (columns) όνομα / επίθετο χρήστη, username, κωδικός χρήστη (password) ορίζοντας τις κατάλληλες εντολές σε γλώσσα ερωτημάτων SQL που απαιτούνται όχι μόνο για τη δημιουργία της βάσης αλλά και του πίνακα όσον αφορά τόσο τον καθορισμό του τύπου δεδομένων των πεδίων του πίνακα όσο και τον ορισμό κατάλληλου πεδίου που θα παίζει το ρόλο πρωτεύοντος κλειδιού αναζήτησης (primary key) εγγραφών (records) στον πίνακα. Δώστε την ενδεδειγμένη λύση για την αποθήκευση του κωδικού του χρήστη στη βάση. Σε γλώσσα προγραμματισμού της επιλογής σας θα δημιουργήσετε μια απλή web εφαρμογή μέσω της οποίας οι χρήστες που δηλώσατε θα μπορούν να κάνουν Login χρησιμοποιώντας κατάλληλο endpoint.

[+15%] **Προαιρετικά:** Ο κώδικάς σας πρέπει να είναι κατάλληλα διαμορφωμένος ώστε να μην δύναται να πραγματοποιηθεί επίθεση SQL injection.

[+15%] **Προαιρετικά:** Δημιουργία login και access μέσω jwt token. Προϋποθέτει την δημιουργία κλειδιών τα οποία θα παράγουν το token το οποίο θα γίνεται assign στο χρήστη και θα λήγει σύμφωνα με το configuration σας

Παραδοτέο:

- Report (max 20 σελίδες)
 - Περιγραφή των απαιτήσεων ασφάλειας
 - Περιγραφή της συνολικής λύσης / σχεδιασμού ασφάλειας και των μηχανισμών /αντιμέτρων που εφαρμόστηκαν ανά απαίτηση
 - Περιγραφή του κρυπτογραφικού πρωτοκόλλου που υλοποιήθηκε, σχόλια επί πληρότητας και ορθότητας του
 - Περιγραφή διαγράμματος ροής (flow chart/diagram)
 - Περιγραφή εργαλείων κώδικα
 - Περιγραφή δομής κώδικα
 - Screenshots εκτέλεσης και επεξήγηση αυτών
 - Περιγραφή εργαλείων αποτίμησης ασφάλειας, αν χρησιμοποιήθηκαν
- Παραδοτέος Κώδικας
 - tar/zip/άλλο με txt αρχείο οδηγιών
- Επίδειξη λειτουργίας

Ομάδες των πολύ δύο φοιτητών

Γλώσσα Προγραμματισμού Backend → Συστήνονται Java, Python, PHP (προτείνουμε τη χρήση του framework java spring boot, python django, php symfony)

DB Backend αν χρειαστεί → επιλογή σας. Συστήνονται MySQL ή MariaDB ή PostgreSQL

Γλώσσα Προγραμματισμού Frontend Περιβάλλον web (όχι mobile app)

Ακολουθεί ένα παράδειγμα διαδικτυακού παίγνιου «κέρματος» που βασίζεται σε συναρτήσεις σύνοψης για υλοποίηση πρωτοκόλλου δέσμευσης (commitment scheme) σε ένα bit (το κέρμα έχει πάντα binary τιμή ως αποτέλεσμα). Ένα commitment scheme είναι ένα κρυπτογραφικό πρωτοκόλλου που επιτρέπει σε ένα μέρος μιας συναλλαγής ή επικοινωνίας να δεσμευτεί σε μια τιμή που έχει επιλεγεί, ή έχει προκύψει (ή σε επιλεγμένη δήλωση) ενώ την κρατά κρυφή σε άλλους, με τη δυνατότητα να αποκαλύψει τη δεσμευμένη τιμή αργότερα. Τα συστήματα δεσμεύσεων έχουν σχεδιαστεί έτσι ώστε κανένα μέρος να μην μπορεί να αλλάξει την τιμή ή τη δήλωση που έχει δεσμευτεί και να μην αμφισβητηθεί η τιμή ή η δήλωση αυτή αφού αποκαλυφθεί.

Bit Commitment – Πως θα υλοποιούσαμε το Coin-flipping ? Εφαρμογή Cryptographic Hash Function

Βήμα 01. **Anna** και **Bill** παράγουν ο κάθε ένας randomly generated string, " r_A " και " r_B ".

Βήμα 02. **Anna** επιλέγει ένα outcome του `flipcoin()`, πχ "head"

Βήμα 03. **Bill** στέλνει σε **Anna** το r_B .

Commits **A**

Βήμα 04. **Anna** υπολογίζει $\text{SHA2}_{256}(\text{"head"} \parallel r_A \parallel r_B) = h_{\text{commit}}$

Βήμα 05. **Anna** στέλνει σε **Bill** το h_{commit}

Sends **B**

Βήμα 06. **Anna** ρωτάει **Bill** : "head or tails"?

Βήμα 07. **Bill** λέει για παράδειγμα "tails".

Opens **A**

Βήμα 08. **Anna** λέει σε **Bill** «κέρδισες»

Βήμα 09. **Anna** στέλνει σε **Bill** το string $(\text{"head"} \parallel r_A \parallel r_B)$

Βήμα 09. **Bill** ελέγχει αν η Άννα είπε αλήθεια // $\text{SHA2}_{256}(\text{"head"} \parallel r_A \parallel r_B) = h_2$

Αλήθεια αν $h_2 == h_{\text{commit}}$